VECTOR DATABASES · NOTEBOOK 0 · START HERE

Part 0 — From Numbers to Vector Databases: Start Here

A guide to Part0_From_Numbers_to_Vector_Databases.ipynb: eight small stages from one number to a working vector database, with one running example — five customer-support documents.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: Python lists · SQLite (built in) · OpenAI `text-embedding-3-small` · Pinecone serverless
Index: `ecom-kb-tutorial` · Namespaces: `support-docs`, `ecom-preview` · No numpy, no maths background

1. Where Part 0 sits
2. The journey, and the class plan
3. Stage 1 — Why we need vector databases
4. Stage 2 — Scalar: one number
5. Stage 3 — Vector: coordinates, then more numbers
6. Stage 4 — Embedding: who makes the numbers
7. Stage 5 — Similarity: one score for closeness
8. Stage 6 — How a vector database works
9. Stage 7 — Hands-on with Pinecone
10. Stage 8 — Before you build
11. Not today
12. Key takeaways, and check yourself
13. Appendix A — Cosine similarity by hand

1 Where Part 0 sits

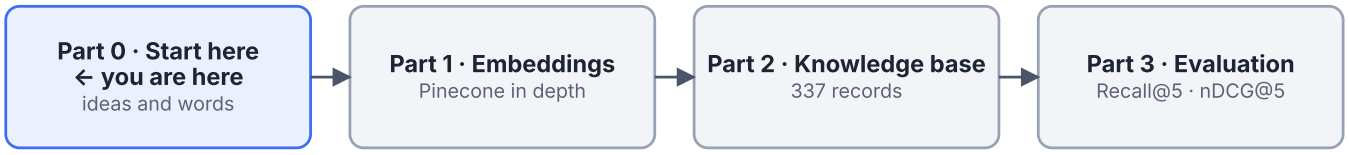


Figure 1. The series. Part 0 builds the ideas; Parts 1–3 use the same Pinecone index.

Part 0 is for students who know Python and SQL but have never met a vector, an embedding or a similarity score. Its job is to make them comfortable with the **words** and the **workflow**. The engineering depth comes in Parts 1–3.

> **The goal of the class.** Every student can explain this sentence in their own words: "A vector database stores numerical representations of information, called embeddings, and finds information by similarity of meaning rather than by exact words."

2 The journey, and the class plan

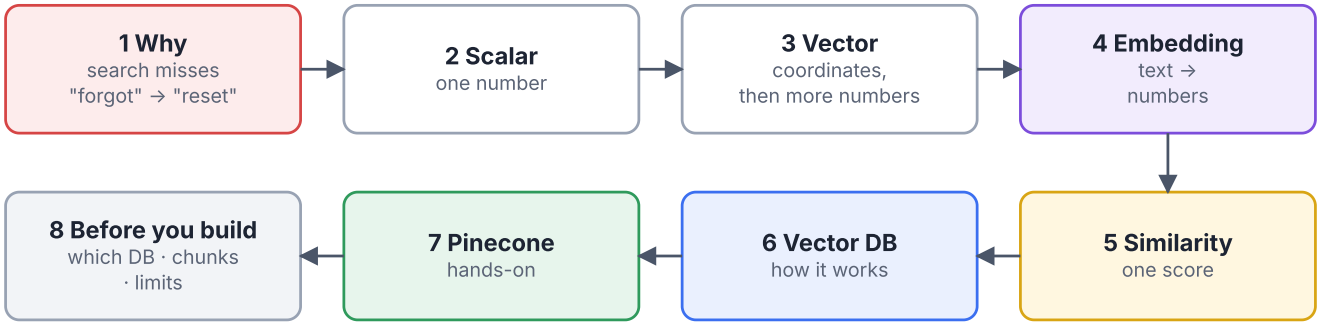


Figure 2. Eight stages. Each one answers the question the previous one raises.

Stage	The question it answers	Time
1 Why	Why can't ordinary search find "reset my password" from "I forgot my password"?	20 min
2–3 Scalar, vector	How do we describe something — a place, a student — with numbers?	35 min
4 Embedding	How do we describe a **sentence** with numbers, and who makes them?	40 min
5 Similarity	How do we tell that two lists of numbers are close?	30 min
6 Vector database	What does it store, what does it return, and how is it different from the model?	50 min
7 Pinecone	How do we do all of this for real?	50 min
8 Before you build	Which database when? Why split documents? What can go wrong?	25 min
Appendix A	Optional: how is cosine similarity calculated?	self-study

About 4½ hours. **One running example** throughout. Every stage ends with a **Check yourself** question (answers in section 12).

3 Stage 1 — Why we need vector databases

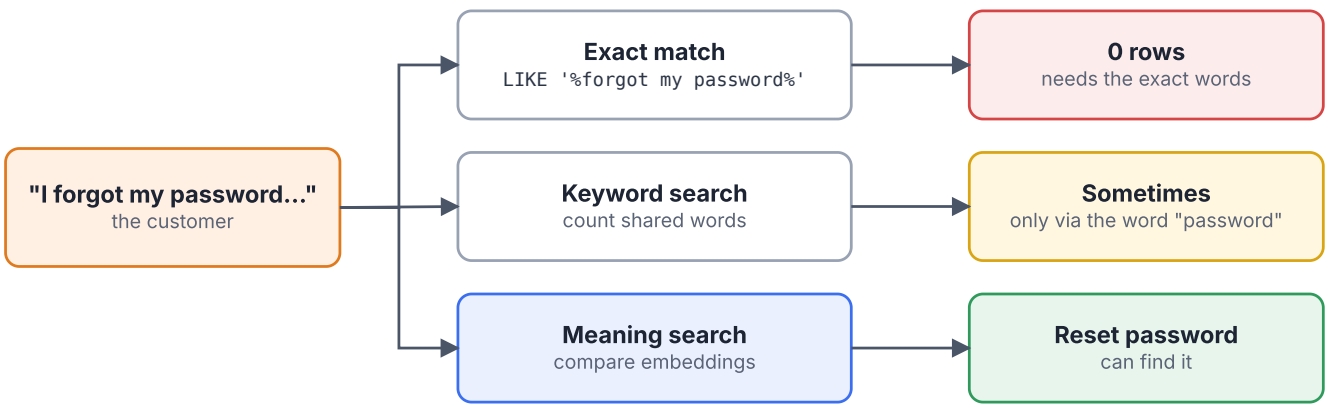


Figure 3. Three kinds of search, one question. Only meaning search connects "forgot" with "reset".

#	Help-centre document	Category
1	How to reset my password?	account
2	How to change my account password?	account
3	How to update my email address?	account
4	How to track my order?	orders
5	How to cancel my subscription?	billing

The notebook tries two ordinary searches before anything new:

Search type	Question	Finds document 1?	Why
Exact match (SQL `LIKE`)	I forgot my password…	**No** — 0 rows	No document has those exact words
Keyword search	I forgot my password…	Sometimes	Only because both say "password" — and it ranks "change my **account** password" higher, sharing more words
Keyword search	I can't log in anymore	**No**	Not one shared word
Meaning search	either	**Can**	Compares meaning, not words (stage 5)

> Be precise with students: ordinary databases are not "dumb". They are excellent at **exact values** (`WHERE order_id = 3001`), and many offer **full-text search** that also matches word forms (reset / resetting). What they don't have is a notion of **meaning**: "forgot", "reset" and "can't log in" are different words to them.

Check yourself 1: "I want to stop paying every month" — which document? Would exact match find it? Keyword search?

4 Stage 2 — Scalar: one number



Figure 4. A scalar is one number. Each box holds exactly one value.

Scalar = one number. Five minutes, no more. It exists so that "vector" has something to be the opposite of.

Check yourself 2: Is `100` a scalar? Is `[100, 4.5]` ?

5 Stage 3 — Vector: coordinates, then more numbers

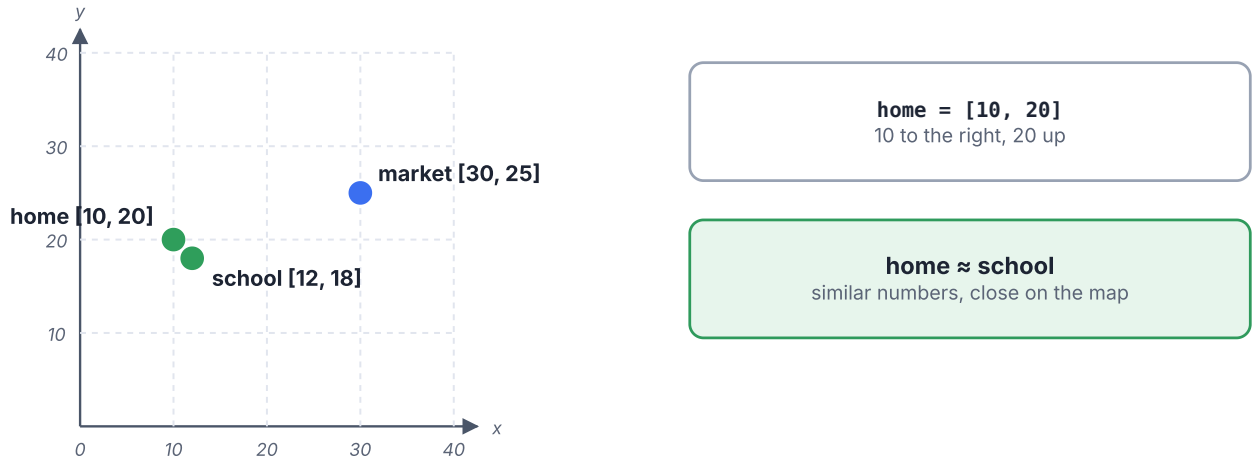


Figure 5. Two numbers identify a place. Similar numbers → nearby places.

Start with a map. One number can't say where something is; two can: **x** = how far right, **y** = how far up. `home = [10, 20]` identifies one place. Home and school have **similar numbers** and are **close** on the map; the market's numbers differ and it is far away. That one observation is the whole idea of vector search — keep coming back to it.

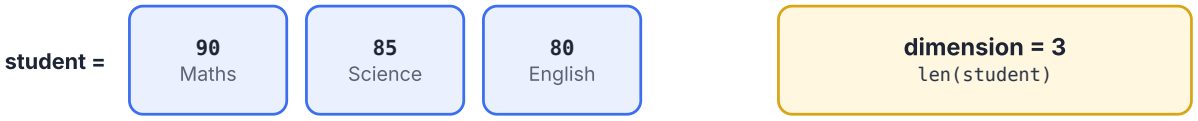


Figure 6. Three numbers describe a student. Dimension = how many numbers.

A student needs three numbers. We can't draw three on paper, but the idea is the same: a position, now with three directions. A list of numbers that describes one thing is a **vector**; how many numbers it has is its **dimension**. In Python a vector is a **list**, and the dimension is `len()`.

Word	Meaning	Example
Scalar	one number	`30`
Vector	a list of numbers that describes one thing	`[10, 20]`, `[90, 85, 80]`
Dimension	how many numbers are in the list	2, 3

> **The bridge to stage 4:** a place needs 2 numbers, a student 3. Could a **sentence** get numbers — hundreds of them — so that similar sentences get similar numbers, like home and school? One difference to flag now: each position in `student` has a **name** (Maths…). In an embedding, the positions won't.

Check yourself 3: How many dimensions do `[10, 20, 30]` and `[100, 4.5, 250, 30, 10]` have?

6 Stage 4 — Embedding: who makes the numbers

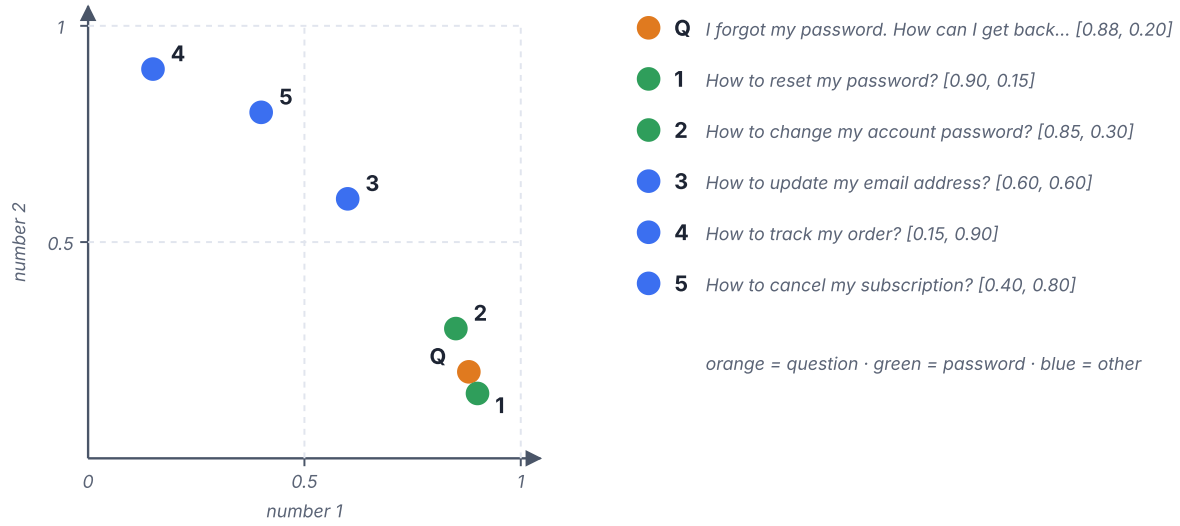


Figure 7. Made-up 2-number positions for our texts. Similar meaning → nearby, like home and school.

Suppose every text could get a position, like a place on a map. Figure 7 shows positions **we chose by hand** to show the idea: the question Q sits next to the two password documents and far from "track my order". Finding the right document would then mean finding the **closest point**.

The obvious question from students: **who decides the numbers?** For seven texts, we did. For millions, a program does — an **embedding model**:

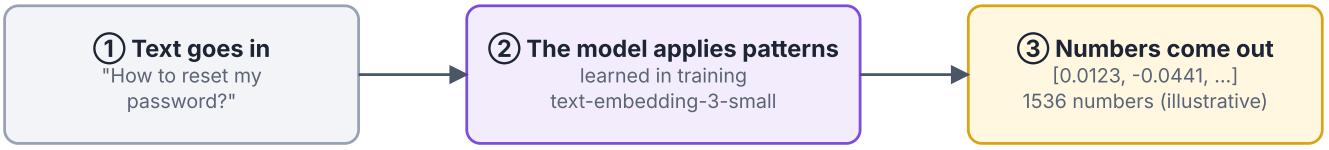


Figure 8. Where real numbers come from: a trained embedding model. We never set them.

OpenAI trained the model on a very large amount of text; it learned which words and phrases are used in similar ways. The list of numbers it returns is the text's **embedding**.

	Student vector (stage 3)	Embedding
Who chooses the numbers	us	the embedding model — **we never set them**
What one position means	a subject (Maths...)	**nothing you can name**; only the whole list means something
Dimension	3	1536 for `text-embedding-3-small`; other models use 384, 768, 1024, 3072...
Short vs long text	—	always the same dimension

> **Say this carefully.** Texts with similar meaning **usually** get similar embeddings — that is what the model learned. It is **not a guarantee**: patterns learned from data can place things in surprising ways. Part 3 measures how often it gets it right. And always **store and search with the same model**: two models give numbers that can't be compared.

The notebook sends the five documents in one call and prints each one's dimension (1536) and first five numbers. No single number is readable — meaning shows up only when two lists are compared.

4c — Tokens, and the input limit



6 tokens (example from OpenAI's tiktoken guide

Figure 9. The model reads tokens — pieces of words — not letters or whole words.

An embedding model doesn't read letters or whole words. It first cuts the text into **tokens** — pieces of words. Common words are usually one token; rare words like "tiktoken" are split into several. OpenAI's `tiktoken` library shows the split (`text-embedding-3-small` uses the `cl100k_base` tokens), and the notebook prints it for our five documents.

Every embedding model has a maximum number of tokens per text, and what happens past it differs:

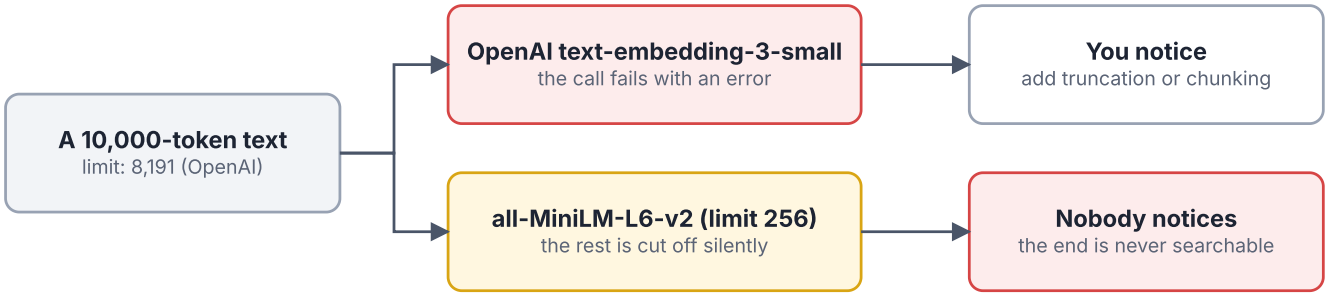


Figure 10. Past the limit, models fail in one of two ways. Chunking avoids both.

Model	Limit per text	Longer text
OpenAI `text-embedding-3-small` (ours)	**8,191 tokens**	the call **fails** — "maximum context length is 8192 tokens". Nothing is cut for you
`all-MiniLM-L6-v2` (popular free model)	**256 word pieces**	the rest is **cut off silently** — no error, no warning

Truncation means cutting the text to the limit yourself before embedding. The error goes away, but whatever came after the cut is **lost**. The notebook proves it: a help manual of over 9,600 tokens with the one useful line — about locked accounts — at the end. OpenAI rejects it; truncated to 8,191 tokens it embeds fine, but the locked-account line is gone, so no customer can ever find it.

Silent truncation is the dangerous one: the pipeline "works", and the end of every long document is simply never searchable. **The real fix is chunking** (stage 8b): split into pieces that each fit, and embed every piece. Per call, OpenAI also allows at most 2,048 texts and about 300,000 tokens in total.

4d — Using an embedding model in practice: five rules

#	Rule	If you forget it
1	**Changing the model means re-embedding everything**	old and new numbers mix; nonsense results, no error
2	**The dimension must match the index**	Pinecone rejects the vectors — or Python silently compares the wrong numbers
3	**The model doesn't know your company's jargon**	"GMV", "AOV", product codes find nothing useful
4	**Cost and size grow with every document**	a surprise bill, a full database
5	**Language: test it, don't assume**	Telugu or Hindi questions miss English documents

Rule 1 — Changing the model means re-embedding everything. Each model has its own map: the same sentence gets completely different numbers from two models. Comparing a question embedded by one model with documents embedded by another gives nonsense — and no error.

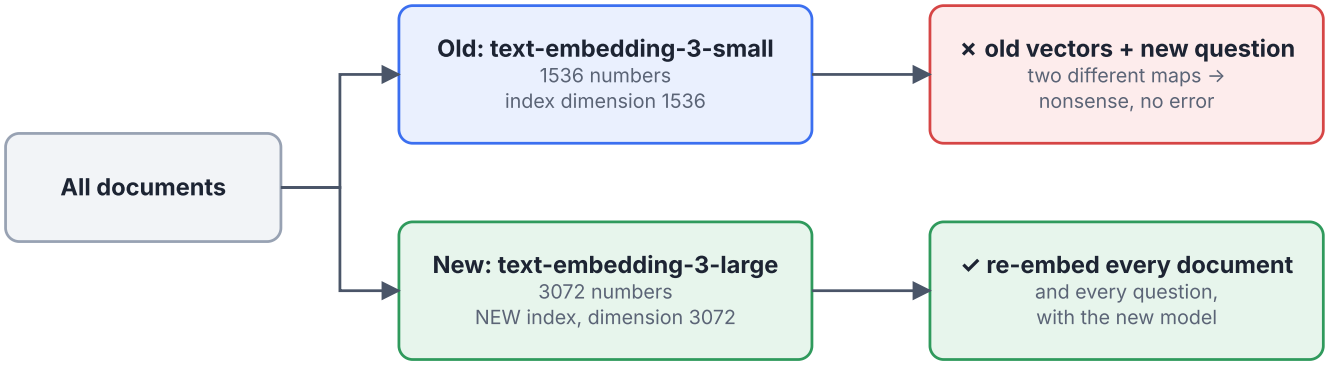


Figure 11. Each model has its own map. Changing the model means a new index and re-embedding everything.

Two habits make this safe: keep the model name in **one** variable (`EMBED_MODEL`) used for documents and questions, and store it with each record (`"embed_model"` in metadata) so you always know which model made a vector.

Rule 2 — The dimension must match the index. An index is created for one dimension and never changes. Ours is 1536:

Model	Dimension	Into our 1536 index
`all-MiniLM-L6-v2`	384	rejected
`text-embedding-3-small`	1536	✓
`text-embedding-3-large`	3072	rejected

> Pinecone refuses a wrong-sized vector — good. Plain Python is the trap: `zip()` stops at the shorter list, so a 1536-number vector against a 384-number one **silently** compares only 384 pairs. The notebook shows it.

Rule 3 — The model doesn't know your company's jargon. It learned general language. "GMV", "AOV", a product code or a project name may mean nothing to it. The fix: **put the meaning into the text you embed.**

Record text embedded	Question "What is our GMV this month?"
total_amount is the money paid for an order.	no shared idea the model knows
total_amount is the money paid for an order. **Also called: revenue, sales, GMV (gross merchandise value).**	the record now contains the users' word

The notebook scores both; the enriched text should win. Part 2 does this for every column with an "Also called" list from the glossary.

Rule 4 — Cost and size. Cost = total tokens × price. Size = vectors × dimension × 4 bytes. Prices as published — check before budgeting; OpenAI's Batch API halves them.

	`text-embedding-3-small`	`text-embedding-3-large`
Price per 1M tokens	$0.02	$0.13 (6.5×)
Size per vector	1536 × 4 bytes ≈ 6 KB	3072 × 4 bytes ≈ 12 KB
Our knowledge base: 337 records × ~60 tokens	$0.0004 · 2.1 MB	$0.0026 · 4.1 MB
A help centre: 1M documents × 200 tokens	$4.00 · 6.1 GB	$26.00 · 12.3 GB

And rule 1 again: **every model change pays the whole embedding cost again.**

Rule 5 — Language: test it, don't assume. Customers write in Telugu, Hindi, English, or mixed ("mera password bhool gaya"). Whether a Telugu question finds an English document depends on the model and how much of that language it saw in training. The notebook scores one meaning written four ways — English, Telugu, Hindi, Hinglish — against "How to reset my password?", next to an unrelated document as the floor. A language scoring near the floor isn't served well: translate the question first, or choose a model built for many languages — and measure the difference with Part 3.

Check yourself 4: Who decides the numbers in an embedding? What does number 37 mean? A 20,000-token manual: what happens with OpenAI? With `all-MiniLM-L6-v2`? You switch to `text-embedding-3-large` — what must you redo?

7 Stage 5 — Similarity: one score for closeness

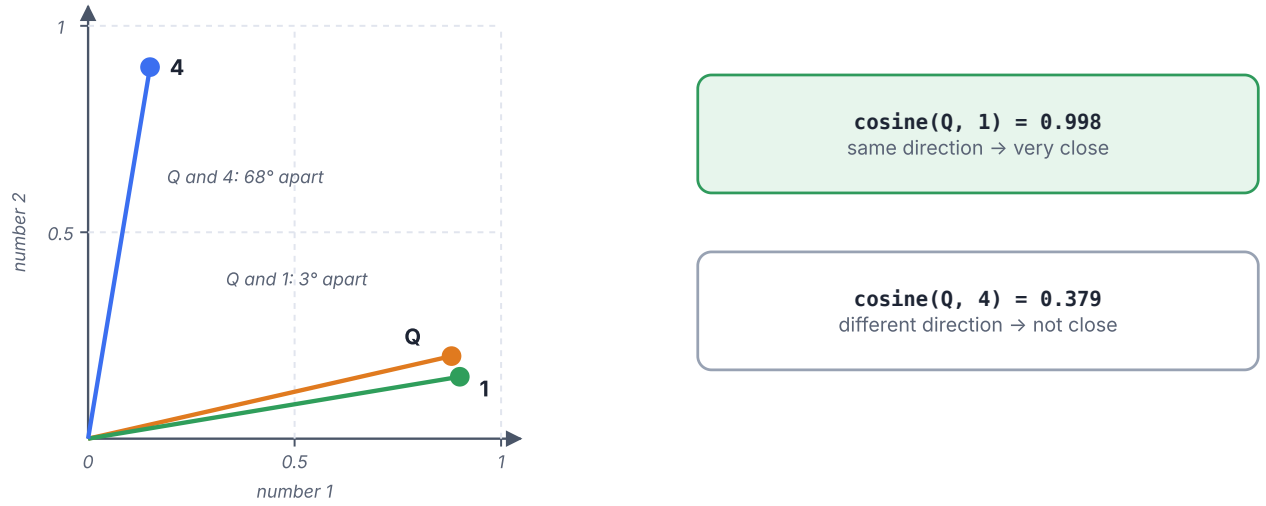


Figure 12. Each vector as an arrow from 0. A small angle between two arrows = similar meaning.

Draw each vector as an **arrow** from 0. Two texts are similar when their arrows point in nearly the **same direction**. Q and "reset password" point almost the same way; Q and "track order" don't.

Document	Angle to Q	Cosine similarity
1 How to reset my password?	3°	0.998
2 How to change my account password?	7°	0.993
3 How to update my email address?	32°	0.846
5 How to cancel my subscription?	51°	0.634
4 How to track my order?	68°	0.379

Cosine similarity turns the angle into one score. Higher = closer. We **sort by it**.

Angle	Cosine similarity	The arrows...
0°	**1**	point the same way
90°	**0**	are at a right angle
180°	**−1**	point opposite ways

> Two cautions for students. **(1)** These describe arrows, not a promise about meaning: a score near 0 doesn't prove two texts are unrelated, and real text embeddings rarely go below 0. **(2)** Real scores bunch together — a good match may score 0.6, not 0.99. Compare scores **with each other**; the order is what matters.

Nobody computes this by hand in practice. The notebook gives students a ready-made `cosine_similarity()` function and Pinecone computes it in stage 7. **Appendix A** works through the maths in seven steps for anyone curious — keep it out of the main class.

Then the real model

The notebook ranks the customer's real question against the five real embeddings, then checks pairs against what students predicted beforehand — including "I can't log in anymore", which shares no words with "I forgot my password".

> The two password documents should come first, though neither says "forgot". Which one wins depends on the model — the question also says "account", and so does document 2.

Check yourself 5: Of "I love programming", "I enjoy coding" and "I like cooking", which two should be most similar? (Students add them to the notebook's pairs and check.)

8 Stage 6 — How a vector database works

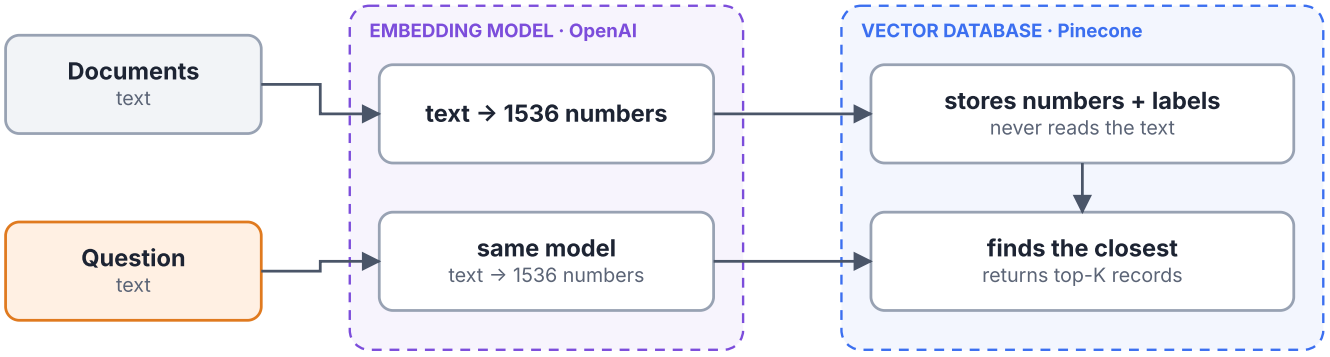


Figure 13. Two different components. The model makes numbers; the database stores and compares them.

6a — The model and the database are two different things

Beginners often think Pinecone "understands" text. It doesn't — it never sees text at all.

	Embedding model	Vector database
Job	turns text into numbers	stores the numbers, finds the closest
Input	text	numbers (+ labels)
Knows anything about meaning?	it **learned** patterns of meaning	no — it only compares numbers
Example	OpenAI `text-embedding-3-small`	Pinecone

The full flow: **1** documents → **2** embedding model → **3** database stores them → **4** a user asks → **5** the **same** model embeds the question → **6** the database finds the closest → **7** we show the matching documents.

6b — What exactly gets stored

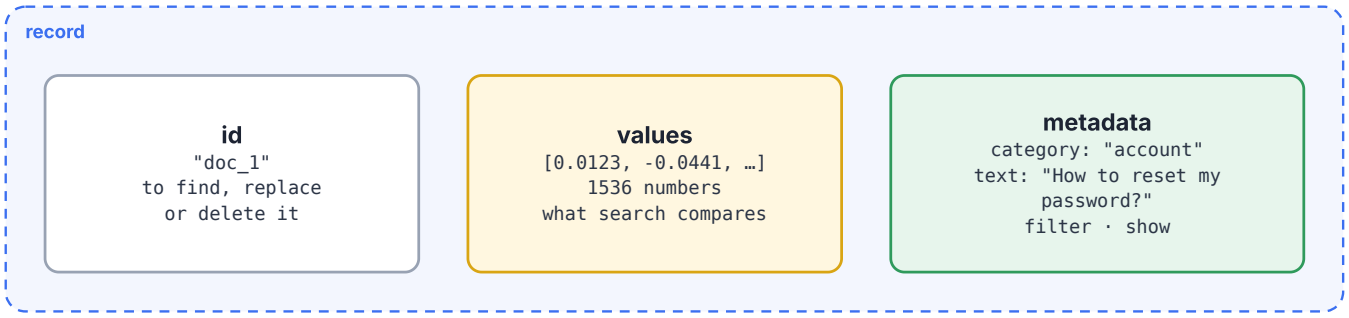


Figure 14. One record: an id, the numbers, and labels. This is all the database keeps.

Field	What it is	Why we need it
`id`	a unique name for the record	to find, replace or delete it later
`values`	the embedding — the numbers	what the search compares
`metadata.category`	our label: account / orders / billing	"search only account documents"
`metadata.text`	the original sentence	numbers can't be turned back into text, so we keep it

Putting the text in metadata is **our choice** for this course. Larger systems often store only an id and fetch the text from another database.

6c — Top-K: how many results come back

Rank	Document	Score (illustrative)
1	How to reset my password?	0.92
2	How to change my account password?	0.88
3	How to update my email address?	0.71
4	How to cancel my subscription?	0.35
5	How to track my order?	0.21

Ask the class: with `top_k=2`, what comes back? (Ranks 1 and 2.) Later, in RAG, the top-K results are what the LLM reads — so K decides how much it gets to see.

6d — A vector database in 20 lines, then the real thing

The notebook builds `TinyVectorDB`: `add()` keeps records in a list; `search()` filters by category, scores every record, sorts, and keeps top-K. Nothing about it is magic.

	TinyVectorDB	Pinecone
Where records live	notebook memory — gone on restart	the cloud — there for every program
Search	scores **every** record	keeps an **index**: vectors organised to find the closest without checking every one

	TinyVectorDB	Pinecone
Size	hundreds	millions
The idea	store → score → sort → top-K	**the same**

The index trades a little exactness for a lot of speed: very fast, and **almost** always the true closest. How it works inside is not needed to use it.

Check yourself 6: What is in one record? What does `top_k=3` return? Does Pinecone turn text into numbers?

9 Stage 7 — Hands-on with Pinecone

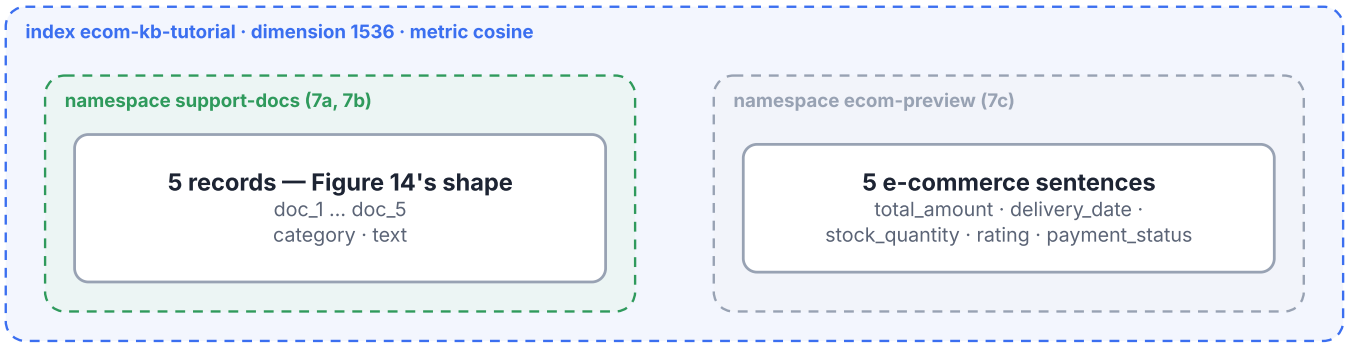


Figure 15. The same index Parts 1–3 use. Part 0 adds two small namespaces.

Pinecone word	Means
Index	the container; dimension (1536) and metric (cosine) are fixed when it is created
Namespace	a separate section inside an index; a search looks in one namespace
Record	`id` + `values` + `metadata` — exactly Figure 14
Upsert	insert, or replace if the id already exists

7a — Store

```
index.upsert(vectors=records, namespace="support-docs")      # records: the shape from 6b
```

The embeddings are the ones from stage 4 — no new OpenAI call. `wait_for()` waits a few seconds until the records are searchable.

7b — Search with top-K, then with a filter

```
index.query(vector=question_vector, top_k=3, namespace="support-docs", include_metadata=True)
index.query(..., filter={"category": {"$eq": "account"}})
```

Put the results next to `TinyVectorDB`'s: same order, same idea. With `metric="cosine"`, Pinecone's `score` **is** the cosine similarity from stage 5.

7c — Same code, e-commerce data

Nothing in the code knows about passwords. The notebook runs the same three steps — embed, upsert, query — on five sentences about our e-commerce database and asks "How much revenue did we make?" and "Which parcels arrived late?". Neither shares its key word with the right sentence: the same situation as "forgot" vs "reset". This is the bridge to Parts 1–3.

Check yourself 7: Index vs record? Namespace vs filter?

10 Stage 8 — Before you build

8a — Which database for which question?

	Relational DB (SQL)	Search engine / full-text	Vector database
Finds	exact values	documents containing the **words**	records with similar **meaning**
Good for	"orders over ₹5,000 in March", totals, joins	"documents mentioning refund policy"	"I forgot my password" → "reset password"
Weak at	meaning	different words for one idea	exact values, ids, totals
Example	PostgreSQL, MySQL	Elasticsearch, OpenSearch	Pinecone

Real systems often use two or three together. In this course the agent uses Pinecone to find the right tables and columns, then SQL to get the exact numbers.

8b — Chunking: why we split long documents

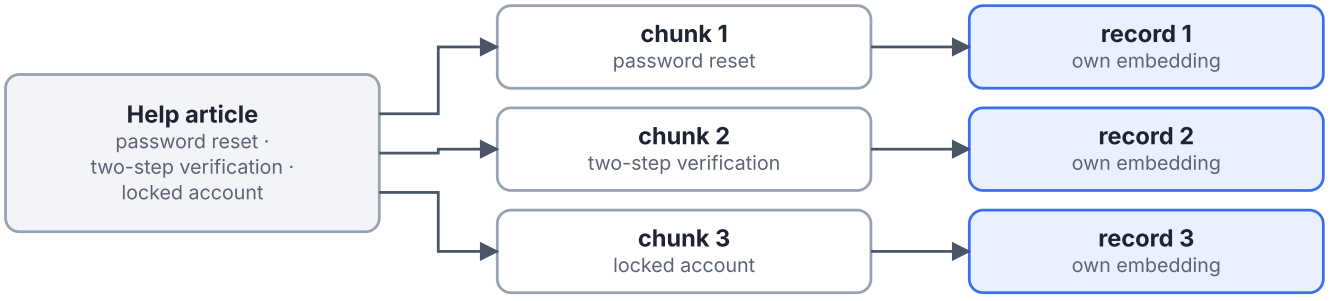


Figure 16. Chunking: one long article about three things becomes three records about one thing each.

Two reasons to split long documents into **chunks**. **(1) The token limit** (4c): past 8,191 tokens a text can't be embedded, and truncating throws the end away. **(2) Blurring:** even under the limit, one embedding for an article about three things is **one** position for **three** topics — a blurred average that matches each topic less well. So each chunk becomes its own record: nothing is lost, and each record is about one thing. The notebook embeds a three-topic article and its three chunks and asks "My account is locked, what do I do?" — students check whether the locked-account chunk beats the whole article. How to split well is a later topic; for now, remember **why**.

8c — The limits of vector search

Limit	What it means
Similar ≠ correct	The closest document can still be the wrong answer
A score is not a probability	0.80 doesn't mean "80% sure"; scores only rank results within one search
It always returns something	Ask about the weather and you still get top-K results
It can miss	The right document may rank 6th when you asked for 5
Quality depends on everything	The model, the data, the chunking and the settings all change results

That is why Part 3 measures search with real questions and known answers.

Check yourself 8: You need "total revenue in March 2024". Vector database, SQL, or both?

11 Not today

Topic	Why it can wait
Matrix maths, linear algebra	Students need only: a vector is a list; a function scores two lists
Deriving cosine similarity	Appendix A, for self-study — not in the main class
How embedding models are trained	We use the model through an API; we don't build it
HNSW / approximate nearest neighbour	How the index is fast inside. Matters for tuning, not for using
How to chunk well	Sizes, overlap, splitting by structure — after the basics

12 Key takeaways, and check yourself

1. Exact match needs the same words; keyword search needs shared words. Neither knows meaning.
2. A **scalar** is one number. A **vector** is a list of numbers describing one thing — like coordinates. **Dimension** = how many numbers.
3. An **embedding model** turns text into a vector (1536 numbers). Similar meaning → **usually** similar vectors. It reads **tokens**, up to a limit (8,191 for ours); past it, an error or a silent cut — so chunk. New model → re-embed everything; dimension must match the index; add your jargon; test your languages.
4. **Cosine similarity**: do the arrows point the same way? Higher = closer. A function computes it; we sort by it.
5. A **vector database** stores records (id + numbers + metadata) and returns the **top-K** closest. It is not the embedding model.
6. Use SQL for exact values, vectors for meaning. Chunk long documents. Similar ≠ correct.

#	Question	Answer
1	"Stop paying every month" — which document? Exact? Keyword?	Cancel subscription. Neither finds it — no shared words

#	Question	Answer
2	`100`? `[100, 4.5]`?	Scalar. Vector with 2 dimensions
3	Dimensions of `[10, 20, 30]`, `[100, 4.5, 250, 30, 10]`?	3 and 5
4	Who decides embedding numbers? Number 37? 20,000-token manual?	The model. Nothing you can name. OpenAI: error; MiniLM: silently uses the first 256 only — chunk it. New model: re-embed all, new 3072 index
5	Programming / coding / cooking — most similar pair?	"Love programming" and "enjoy coding"
6	In a record? `top_k=3`? Does Pinecone embed?	id, values, metadata. The 3 best. No — the model does
7	Index vs record? Namespace vs filter?	Container vs one item. A section you store into vs a rule on one search
8	Revenue in March 2024?	Both: vectors find where revenue lives; SQL adds it up

Explain it back: "A vector database stores numerical representations of information, called embeddings, and finds information by similarity of meaning rather than by exact words." **Next — Part 1.**

13 Appendix A — Cosine similarity by hand

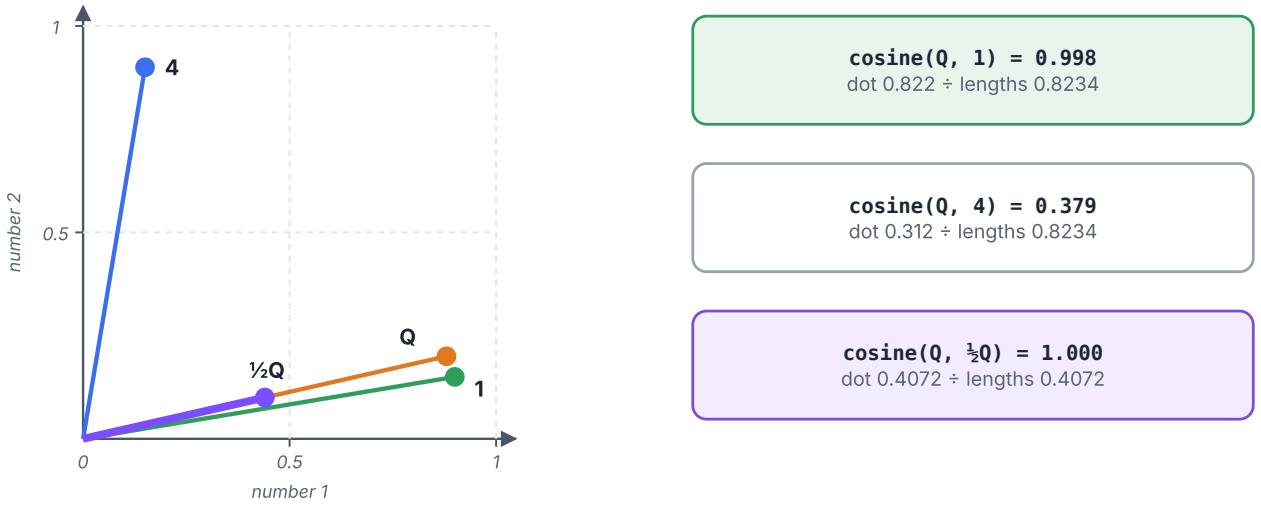


Figure 17. Q, ½Q, 1 and 4 as arrows. ½Q is shorter but points exactly where Q points.

Optional. For students who want to see what `cosine_similarity()` does. The notebook's Appendix A cell prints every step below for every document.

Steps 1–3 — Point, arrow, direction

A vector is a point (stage 3); draw it as an arrow from 0 (stage 5). Halve Q: ½Q = `[0.44, 0.10]` — shorter, same direction. A good score must give Q and ½Q the **top** score: direction matters, length

doesn't.

Step 4 — Multiply matching numbers, then add (the dot product)

	Number 1	Number 2	Add
Q × 1	0.88 × 0.90 = 0.792	0.20 × 0.15 = 0.030	**0.822**
Q × 4	0.88 × 0.15 = 0.132	0.20 × 0.90 = 0.180	**0.312**

Big × big in the same position gives a big result. **Problem:** double Q to `[1.76, 0.40]` and the dot product with document 1 doubles to 1.644, though the direction didn't change. Length must come out.

Step 5 — The length of each arrow

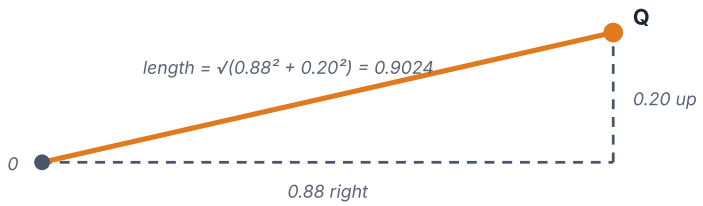


Figure 18. Q goes 0.88 right and 0.20 up. Its length is the long side of that triangle.

Arrow	Square each number	Add	√ = length
Q	$0.88^2 = 0.7744 \cdot 0.20^2 = 0.0400$	0.8144	**0.9024**
1	$0.90^2 = 0.8100 \cdot 0.15^2 = 0.0225$	0.8325	**0.9124**
4	$0.15^2 = 0.0225 \cdot 0.90^2 = 0.8100$	0.8325	**0.9124**

Keep four decimals — rounding too early changes the last digit.

Step 6 — Divide

```
cosine similarity  =  dot product  ÷  (length of Q  ×  length of doc)
```

Document	Dot	Lengths	Cosine
1	0.822	0.9024 × 0.9124 = 0.8234	**0.998**
4	0.312	0.9024 × 0.9124 = 0.8234	**0.379**
½Q	0.4072	0.9024 × 0.4512 = 0.4072	**1.000**

½Q gets the top score: length is gone, only direction is left. The result is the cosine of the angle (cos 3.3° = 0.998), hence the name.

Step 7 — Three metrics, one order

Document	Cosine (higher)	Dot product (higher)	Distance between tips (lower)
1 How to reset my password?	0.998	0.822	0.054
2 How to change my account password?	0.993	0.808	0.104
3 How to update my email address?	0.846	0.648	0.488
5 How to cancel my subscription?	0.634	0.512	0.768
4 How to track my order?	0.379	0.312	1.011

OpenAI makes every arrow **length 1**. Step 6 then divides by 1 × 1, so cosine **is** the dot product; and with equal lengths a smaller angle means closer tips, so distance gives the same order. Made length 1, Q = [0.975, 0.222] and document 1 = [0.986, 0.164]: 0.975 × 0.986 + 0.222 × 0.164 = 0.998.